

Assignment :1.

Q2. Consider a system with ‘n’ processes and ‘m’ resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not. (Use resource request algorithm).

```
#include <stdio.h>
int main()
{
    int n, m;
    int i, j, p;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int total[m], available[m];
    int allocation[n][m], maximum[n][m], need[n][m];
    int request[m];
    // Accept total instances of each resource
    printf("\nEnter total instances of each resource:\n");
    for (j = 0; j < m; j++)
    {
        printf("Resource R%d: ", j);
        scanf("%d", &total[j]);
    }
    // Accept Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for (i = 0; i < n; i++)
    {
        printf("Process P%d: ", i);
        for (j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }
    // Accept Maximum Matrix
    printf("\nEnter Maximum Requirement Matrix:\n");
    for (i = 0; i < n; i++)
    {
        printf("Process P%d: ", i);
        for (j = 0; j < m; j++)
        {
            scanf("%d", &maximum[i][j]);
        }
    }
    // Calculate Need Matrix
    // Need = Maximum - Allocation
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}
```

```

}
// Calculate Available resources
for (j = 0; j < m; j++)
{
    available[j] = total[j];
    for (i = 0; i < n; i++)
    {
        available[j] -= allocation[i][j];
    }
}

// Display Need Matrix
printf("\n-----");
printf("\n    NEED MATRIX");
printf("\n-----\n");
for (i = 0; i < n; i++)
{
    printf("P%d\t", i);
    for (j = 0; j < m; j++)
    {
        printf("%d\t", need[i][j]);
    }
    printf("\n");
}
// Display Available Resources
printf("\nAvailable Resources:\n");
for (j = 0; j < m; j++)
{
    printf("R%d = %d\t", j, available[j]);
}
// Accept process number
printf("\n\nEnter process number making the request (0 to %d): ", n - 1);
scanf("%d", &p);
// Accept request
printf("Enter resource request for P%d:\n", p);
for (j = 0; j < m; j++)
{
    printf("Request R%d: ", j);
    scanf("%d", &request[j]);
}
// Step 1: Check Request <= Need
for (j = 0; j < m; j++)
{
    if (request[j] > need[p][j])
    {
        printf("\nRequest cannot be granted.");
        printf("\nReason: Process P%d has exceeded its maximum claim.\n", p);
        return 0;
    }
}
// Step 2: Check Request <= Available
for (j = 0; j < m; j++)
{
    if (request[j] > available[j])
    {

```

```

        printf("\nRequest cannot be granted immediately.");
        printf("\nReason: Resources are not currently available.\n");
        return 0;
    }
}
int work[m];
int finish[n];
// Temporary allocation
for (j = 0; j < m; j++)
{
    work[j] = available[j] - request[j];
}
int tempAllocation[n][m];
int tempNeed[n][m];
for (i = 0; i < n; i++)
{
    finish[i] = 0;

    for (j = 0; j < m; j++)
    {
        tempAllocation[i][j] = allocation[i][j];
        tempNeed[i][j] = need[i][j];
    }
}
// Pretend to grant request
for (j = 0; j < m; j++)
{
    tempAllocation[p][j] += request[j];
    tempNeed[p][j] -= request[j];
}
// Safety algorithm
int count = 0;
int found;
while (count < n)
{
    found = 0;
    for (i = 0; i < n; i++)
    {
        if (finish[i] == 0)
        {
            int possible = 1;
            for (j = 0; j < m; j++)
            {
                if (tempNeed[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if (possible)
            {
                // Process can finish
                for (j = 0; j < m; j++)
                {

```

```

        work[j] += tempAllocation[i][j];
    }
    finish[i] = 1;
    count++;
    found = 1;
}
}
}
// No process can finish -> unsafe state
if (found == 0)
{
    break;
}
}
// Final result
if (count == n)
{
    printf("\n-----");
    printf("\nREQUEST CAN BE GRANTED");
    printf("\n-----\n");
    printf("The system remains in a SAFE state.\n");
    printf("Request of Process P%d can be granted immediately.\n", p);
}
else
{
    printf("\n-----");
    printf("\nREQUEST CANNOT BE GRANTED");
    printf("\n-----\n");
    printf("Granting the request would result in an UNSAFE state.\n");
    printf("Therefore, the request is not granted.\n");
}
return 0;
}

```

Q2.Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call child process sorts the integers using insertion sort.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void bubbleSort(int a[], int n)
{
    int i, j, temp;

```

```

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

void insertionSort(int a[], int n)
{
    int i, j, key;
    for (i = 1; i < n; i++)
    {
        key = a[i];
        j = i - 1;

        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

void display(int a[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
}

int main()
{
    int n, i;
    int a[100], parentArray[100], childArray[100];
    pid_t pid;
    printf("Enter number of integers: ");
    scanf("%d", &n);
    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    /* Create copies before fork */
    for (i = 0; i < n; i++)
    {

```

```

        parentArray[i] = a[i];
        childArray[i] = a[i];
    }
    pid = fork();
    if (pid < 0)
    {
        printf("Fork failed.\n");
        return 1;
    }
    else if (pid == 0)
    {
        /* Child process */
        printf("\nChild Process (Insertion Sort)\n");
        insertionSort(childArray, n);
        printf("Sorted array by Insertion Sort: ");
        display(childArray, n);
        exit(0);
    }

    else
    {
        /* Parent process */
        printf("\nParent Process (Bubble Sort)\n");
        bubbleSort(parentArray, n);
        printf("Sorted array by Bubble Sort: ");
        display(parentArray, n);

        /* Wait for child process */
        wait(NULL);
        printf("Parent process: Child process completed.\n");
    }
    return 0;
}

```

Q.3 Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice system call.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
int main()
{
    pid_t pid;
    int status;
    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());
    printf("Parent Nice value: %d\n", nice(0));

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
    }
}

```

```

    exit(1);
}
if (pid == 0)
{
    /* Child process */
    printf("\nChild process started.\n");
    printf("Child PID: %d\n", getpid());
    printf("Child Nice value before change: %d\n", nice(0));
    /* Decrease nice value to increase priority */
    errno = 0;
    if (nice(-5) == -1 && errno != 0)
    {
        perror("nice");
        printf("Run the program with appropriate privileges.\n");
        exit(1);
    }
    printf("Child Nice value after change: %d\n", nice(0));
    printf("Child process is given higher priority.\n");
    exit(0);
}
else
{
    /* Parent process */
    wait(&status);
    printf("\nParent process: Child has completed.\n");
    printf("Parent PID: %d\n", getpid());
}
return 0;
}

```